Name : Parth Pankaj Thikrul PRN : 25070122204 Batch : C-C1

EXPERIMENT 11

Create a MongoDB Atlas cluster, connect it to a web application, and perform basic CRUD operations

Course Outcome	CO5	**Teaching Hours**	2

Objective

To provision a free MongoDB Atlas cloud cluster, connect a Node.js/Express web application to it using Mongoose, and implement CRUD operations through REST API routes.

Theory

MongoDB Atlas is MongoDB's fully-managed cloud database service, removing the need to self-host a database server. Connecting requires: creating an Atlas account and a free-tier (M0) cluster, configuring Network Access (IP allowlist) so your machine can reach the cluster, creating a Database User with credentials, and obtaining a connection string (SRV URI). Mongoose is a popular Object Data Modeling (ODM) library for Node.js that maps MongoDB documents to JavaScript Schemas/Models, adding validation and simplifying queries compared to the raw MongoDB driver.

Software / Hardware Requirements

A free MongoDB Atlas account (mongodb.com/cloud/atlas), Node.js, npm packages: express, mongoose, dotenv.

Step-by-Step Procedure

1. Sign up / log in at mongodb.com/cloud/atlas and create a new free-tier Project.

2. Inside the project, click 'Build a Database', choose the M0 Free tier, select a cloud provider/region close to you, and create the cluster (name it e.g. Cluster0).

3. Under Security > Database Access, create a new Database User with a username and a strong auto-generated password; note these credentials securely.

4. Under Security > Network Access, add an IP entry — for lab purposes select 'Allow Access from Anywhere' (0.0.0.0/0), noting this is not recommended for production.

5. Once the cluster is provisioned, click 'Connect' > 'Drivers', select Node.js, and copy the provided connection string (mongodb+srv://<username>:<password>@cluster0...).

6. Create a project folder lab14_atlas, run npm init -y, then install dependencies.

```
npm install express mongoose dotenv
```

7. Create a .env file storing the Atlas connection string (replacing <password> with the actual password, and adding a database name at the end of the URI, e.g. /labDB).

```
MONGO_URI=mongodb+srv://labuser:yourpassword@cluster0.xxxxx.mongodb.net/labDB?retryWrites=true&w=majority
```

8. Create server.js: load env vars, connect to Atlas using mongoose.connect(), and start an Express server.

```
// server.js
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('Connected to MongoDB Atlas'))
  .catch(err => console.error('Connection error:', err));

app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

9. Run node server.js and confirm the console prints 'Connected to MongoDB Atlas' (also visible as an active connection in the Atlas dashboard's Metrics tab).

10. Define a Mongoose Schema and Model for a Task resource in a new file models/Task.js.

```
// models/Task.js
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  title: { type: String, required: true },
  completed: { type: Boolean, default: false },
```

```
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Task', taskSchema);
```

11. Back in server.js, require the Task model and implement 4 REST routes for full CRUD: POST /tasks, GET /tasks, PUT /tasks/:id, DELETE /tasks/:id.

```
const Task = require('./models/Task');

// CREATE
app.post('/tasks', async (req, res) => {
  const task = await Task.create(req.body);
  res.status(201).json(task);
});

// READ
app.get('/tasks', async (req, res) => {
  const tasks = await Task.find();
  res.json(tasks);
});

// UPDATE
app.put('/tasks/:id', async (req, res) => {
  const updated = await Task.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(updated);
});

// DELETE
app.delete('/tasks/:id', async (req, res) => {
  await Task.findByIdAndDelete(req.params.id);
```

```
  res.json({ message: 'Task deleted' });
});
```

12. Restart the server and test each route using Postman (or curl): create 2-3 tasks with POST, list them with GET, update one's 'completed' field to true with PUT, and delete one with DELETE.

13. Open the Atlas dashboard, navigate to Browse Collections for your cluster, and visually confirm the labDB.tasks collection reflects the changes made via the API in real time.

Hands-on Assignment

Index.html

public > index.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Student Management System</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
        }

        h1 {
            color: #333;
        }

        input {
            padding: 10px;
            margin: 5px;
            width: 200px;
        }

        button {
            padding: 10px 15px;
            margin: 5px;
            cursor: pointer;
        }

```

public > index.html > html > body

```
<html>
<head>
    <style>
        table {
            border-collapse: collapse;
            width: 80%;
            margin-top: 30px;
        }

        th, td {
            border: 1px solid #999;
            padding: 10px;
            text-align: left;
        }

        th {
            background-color: #eee;
        }
    </style>
</head>
<body>
    <h1>Student Management System</h1>
    <h2>Add Student</h2>
    <input
        type="text"
        id="name"
```

public > index.html > html > body

```
<html>
<body>
    <input
        placeholder="Student Name"
    >
    <input
        type="text"
        id="rollNo"
        placeholder="Roll Number"
    >
    <input
        type="text"
        id="course"
        placeholder="Course"
    >
    <button onclick="addStudent()">
        Add Student
    </button>
    <h2>Student List</h2>
    <button onclick="getStudents()">
        Refresh Students
    </button>
    <table>
        <thead>
            <tr>
                <th>Name</th>
```

public > index.html > html > body

```
<html>
<body>
    <table>
        <thead>
            <tr>
                <th>Name</th>
                <th>Roll Number</th>
                <th>Course</th>
                <th>Action</th>
            </tr>
        </thead>
        <tbody id="studentTable">
        </tbody>
    </table>
    <script>
        // ====================================
        // CREATE
        // ====================================
        async function addStudent() {

            const name =
                document.getElementById("name").value;

            const rollNo =
                document.getElementById("rollNo").value;

```

public > index.html > html > body

```
<html>
<body>
    <script>
        async function addStudent() {

            const course =
                document.getElementById("course").value;


            const student = {
                name: name,
                rollNo: rollNo,
                course: course
            };
            const response = await fetch("/students", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },

                body: JSON.stringify(student)
            });
            const result = await response.json();
            alert(result.message);
            document.getElementById("name").value = "";
            document.getElementById("rollNo").value = "";
```

```
public > index.html > html > body
  2   <html>
 44   <body>
 81       <script>
 85           async function addStudent() {
113               document.getElementById("rollNo").value = "";
114               document.getElementById("course").value = "";
115               getStudents();
116
117           }
118
119
120           // ====================================
121           // READ
122           // ====================================
123
124           async function getStudents() {
125
126               const response =
127                   await fetch("/students");
128               const students =
129                   await response.json();
130               const table =
131                   document.getElementById("studentTable");
132               table.innerHTML = "";
133               students.forEach(student => {
134                   const row = `
```

```
public > index.html > html > body
  2   <html>
 44   <body>
 81       <script>
124           async function getStudents() {
133               students.forEach(student => {
134                   const row = `
135
136                       <tr>
137                           <td>
138                               ${student.name}
139                           </td>
140                           <td>
141                               ${student.rollNo}
142                           </td>
143                           <td>
144                               ${student.course}
145                           </td>
146                           <td>
147
148                               <button
149                                   onclick="deleteStudent('${student._id}')"
150                               >
151                                   Delete
152                               </button>
153                           </td>
154                       </tr>
```

```
public > index.html > html > body
  2   <html>
 44   <body>
 81       <script>
124           async function getStudents() {
133               students.forEach(student => {
154                       </tr>
155
156                   `;
157                   table.innerHTML += row;
158               });
159           }
160
161
162           // ====================================
163           // DELETE
164           // ====================================
165
166           async function deleteStudent(id) {
167
168               const response =
169                   await fetch(`/students/${id}`, {
170                       method: "DELETE"
171                   });
172               const result =
173                   await response.json();
174               alert(result.message);
```

```
public > index.html > html > body
  2   <html>
 44   <body>
 81       <script>
124           async function getStudents() {
158               });
159           }
160
161
162           // ====================================
163           // DELETE
164           // ====================================
165
166           async function deleteStudent(id) {
167
168               const response =
169                   await fetch(`/students/${id}`, {
170                       method: "DELETE"
171                   });
172               const result =
173                   await response.json();
174               alert(result.message);
175               getStudents();
176           }
177       </script>
178   </body>
179   </html>
```

Server.js

server.js > connectDB > app.listen() callback

```
const express = require("express");
const { MongoClient, ObjectId } = require("mongodb");
const dns = require("dns"); // 1. Import node's dns module
require("dotenv").config();

dns.setServers(["8.8.8.8", "8.8.4.4"]);

const app = express();
const PORT = 3000;

// Middleware
app.use(express.json());
app.use(express.static("public"));

// MongoDB connection
const client = new MongoClient(process.env.MONGODB_URI, {
    family: 4
});

let studentsCollection;

// Connect to MongoDB
async function connectDB() {

    try {

```

server.js > connectDB > app.listen() callback

```
async function connectDB() {
        await client.connect();

        const db = client.db("collegeDB");

        studentsCollection = db.collection("students");

        console.log("Connected to MongoDB Atlas");

        // Start server only after MongoDB connection
        app.listen(PORT, () => {
            console.log(`Server running at http://localhost:${PORT}`);
        });

    } catch (error) {

        console.error("MongoDB connection error:", error);

    }
}

connectDB();


// ========================================
// CREATE - Add Student
```

server.js > connectDB > app.listen() callback

```
52  // ========================================
53
54  app.post("/students", async (req, res) => {
55
56      try {
57
58          const student = req.body;
59
60          const result =
61              await studentsCollection.insertOne(student);
62
63          res.json({
64
65              message: "Student added successfully",
66
67              id: result.insertedId
68
69          });
70
71      } catch (error) {
72
73          res.status(500).json({
74
75              error: error.message
76
77          });
```

server.js > app.post("/students") callback

```
83
84  // ========================================
85  // READ - Get All Students
86  // ========================================
87
88  app.get("/students", async (req, res) => {
89
90      try {
91
92          const students =
93              await studentsCollection.find().toArray();
94
95          res.json(students);
96
97      } catch (error) {
98
99          res.status(500).json({
100
101             error: error.message
102
103         });
104
105     }
106
107 });
108
```

```
// ========================================
// DELETE - Delete Student
// ========================================

app.delete("/students/:id", async (req, res) => {

    try {

        const id = req.params.id;

        const result =
            await studentsCollection.deleteOne({

                _id: new ObjectId(id)

            });

        res.json({

            message: "Student deleted successfully",

            deletedCount: result.deletedCount

        });

    } catch (error) {
```

```
            });

        res.json({

            message: "Student deleted successfully",

            deletedCount: result.deletedCount

        });

    } catch (error) {

        res.status(500).json({

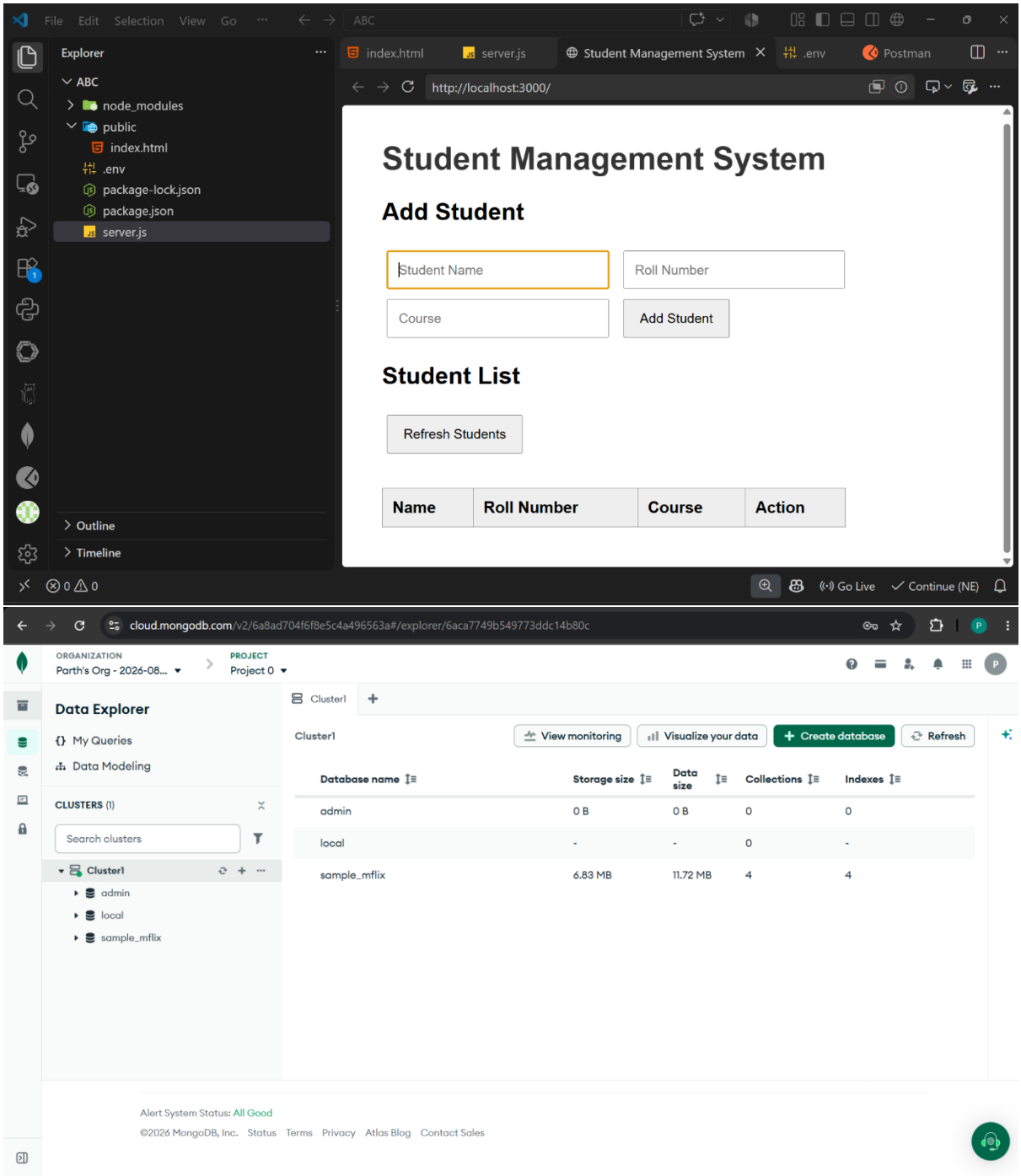
            error: error.message

        });

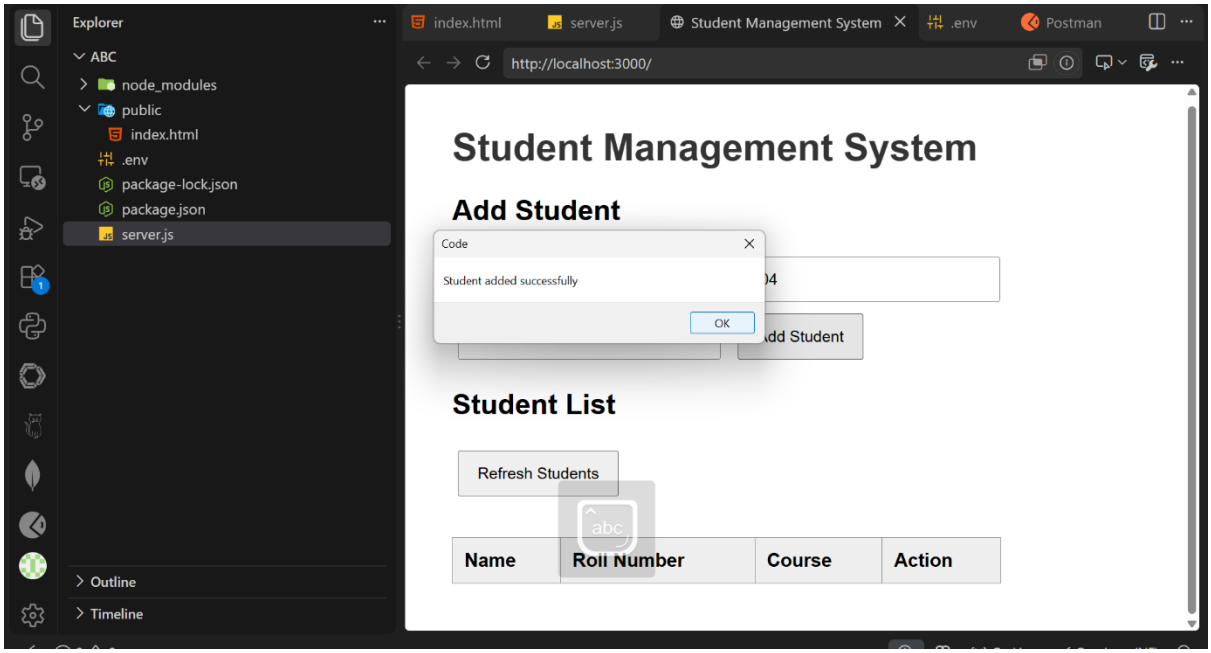
    }

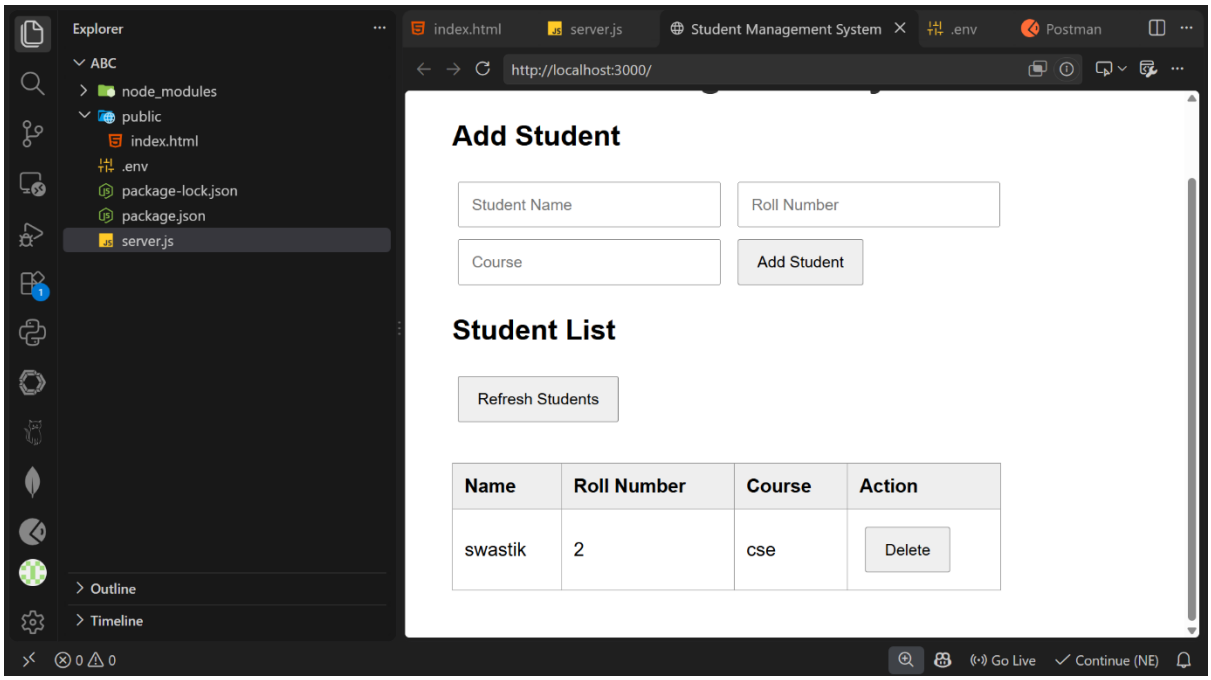
});
```

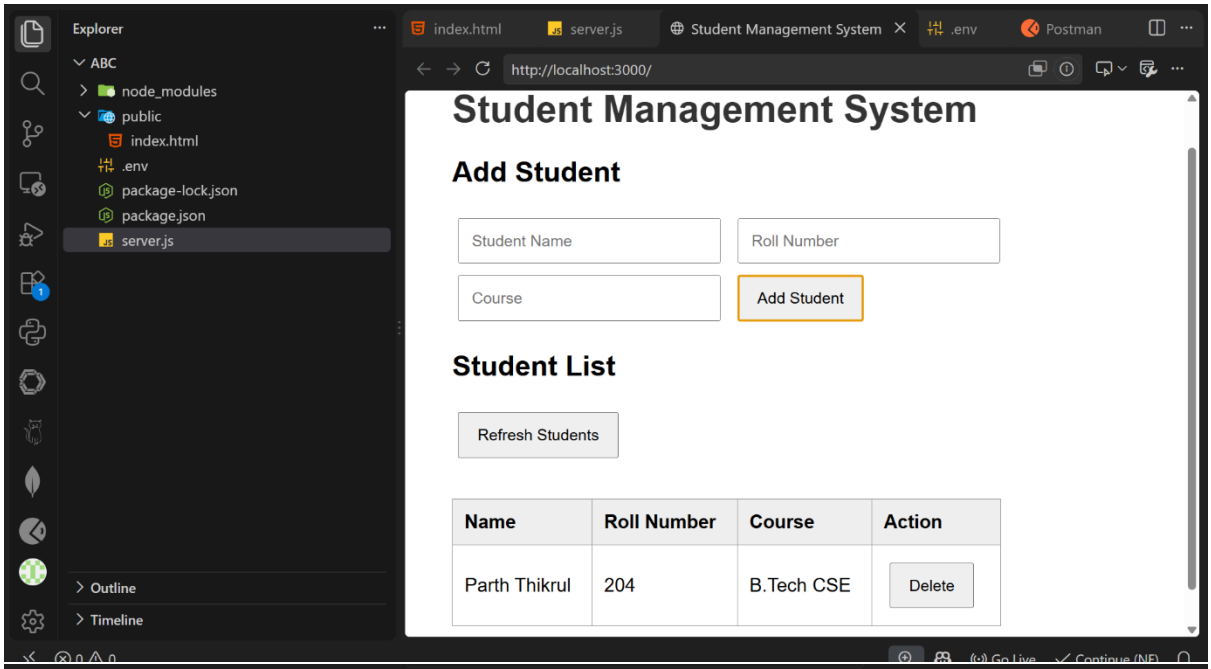
Output

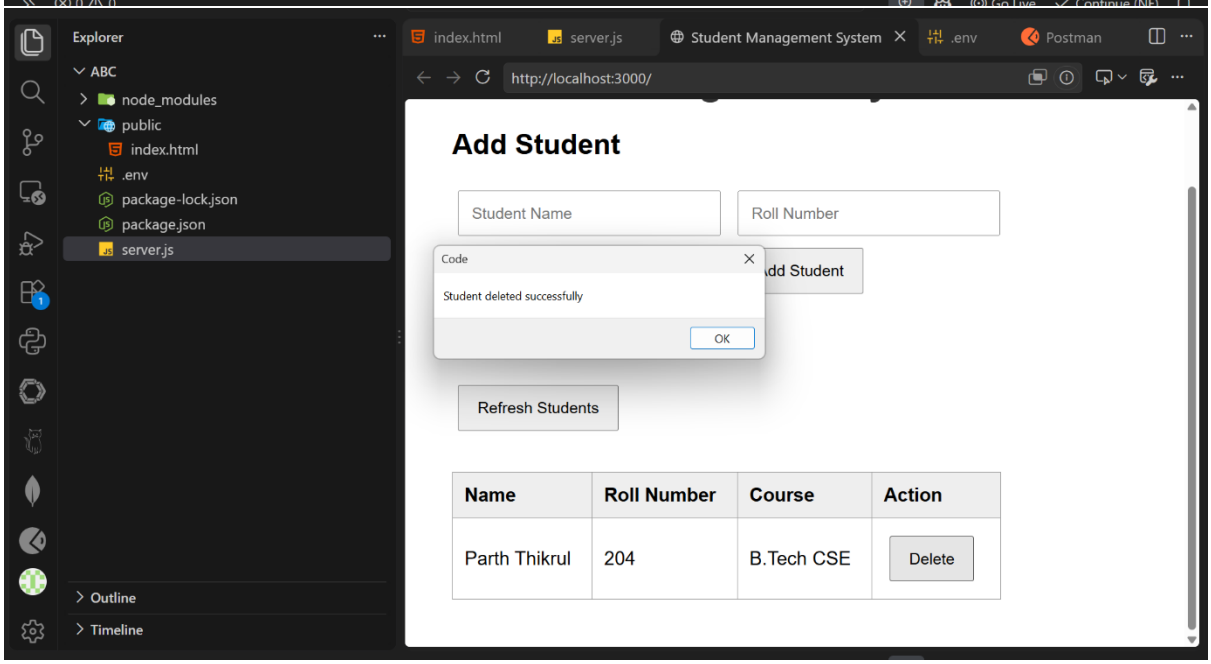

File Edit Selection View Go
Explorer
ABC
node_modules
public
index.html
.env
package-lock.json
package.json
server.js
Outline
Timeline
index.html
server.js
Student Management System
.env
Postman
http://localhost:3000/
Student Management System
Add Student
Student Name
Roll Number
Course
Add Student
Student List
Refresh Students
Name
Roll Number
Course
Action
Go Live
Continue (NE)

cloud.mongodb.com/v2/6a8ad704f6f8e5c4a496563a#/explorer/6aca7749b549773ddc14b80c
ORGANIZATION
Parth's Org - 2026-08...
PROJECT
Project 0
Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
Cluster1
admin
local
sample_mflix
Cluster1
View monitoring
Visualize your data
Create database
Refresh
Database name
Storage size
Data size
Collections
Indexes
admin 0 B 0 B 0 0
local - - 0 -
sample_mflix 6.83 MB 11.72 MB 4 4
Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales


Explorer
ABC
node_modules
public
index.html
.env
package-lock.json
package.json
server.js
Outline
Timeline
index.html
server.js
Student Management System
.env
Postman
http://localhost:3000/
Student Management System
Add Student
Code
Student added successfully
OK
Add Student
Student List
Refresh Students
Name
Roll Number
Course
Action


Explorer
ABC
node_modules
public
index.html
.env
package-lock.json
package.json
server.js
Outline
Timeline
index.html
server.js
Student Management System
.env
Postman
http://localhost:3000/
Add Student
Student Name
Roll Number
Course
Add Student
Student List
Refresh Students
Name
Roll Number
Course
Action
swastik
2
cse
Delete
Go Live
Continue (NE)


Explorer
ABC
node_modules
public
index.html
.env
package-lock.json
package.json
server.js
Outline
Timeline
index.html
server.js
Student Management System
.env
Postman
http://localhost:3000/
Student Management System
Add Student
Student Name
Roll Number
Course
Add Student
Student List
Refresh Students
Name
Roll Number
Course
Action
Parth Thikrul
204
B.Tech CSE
Delete


Explorer
ABC
node_modules
public
index.html
.env
package-lock.json
package.json
server.js
Outline
Timeline
index.html
server.js
Student Management System
.env
Postman
http://localhost:3000/
Add Student
Student Name
Roll Number
Code
Student deleted successfully
OK
Refresh Students
Name
Roll Number
Course
Action
Parth Thikrul
204
B.Tech CSE
Delete